Lecture 26.08: Gradient Descent vs OLS Comparison

📌 Topic Overview

In this lecture, Shraddha ma'am compares **Gradient Descent** and **Ordinary Least Squares (OLS)** head-to-head across computational complexity, memory scaling, and real-world production choice.

⚖️ Head-to-Head Architectural Comparison

Dimension	Ordinary Least Squares (OLS)	Gradient Descent (GD)
Formula	$\theta = (X^T X)^{-1} X^T y$	$w := w - \alpha \nabla J$
Computational Complexity	$\mathcal{O}(d^3)$ due to matrix inversion	$\mathcal{O}(k \cdot m \cdot d)$ linear in features
Hyperparameter Tuning	**Zero** tuning required	Must tune α (learning rate) & epochs
Memory Requirement	High (must invert $d \times d$ matrix)	Low (only stores weight vector)
Best Used When	Number of features $d < 10,000$	Large features $d > 10,000$ or Big Data streams

---

📊 Decision Flowchart: GD vs OLS



💻 Python Implementation

```
def compare_gd_vs_ols(n_samples, n_features):
    """
    Recommends whether to use Gradient Descent or OLS (Normal Equation)
    based on sample size (m) and feature dimension (d).

    Complexity Rules:
    - OLS:              O(d^3) computation, memory heavy for large d
    - Gradient Descent: O(k * m * d), highly scalable to Big Data
    """
    if n_features > 10000:
        recommendation = "Gradient Descent (OLS matrix inversion O(d^3) will crash
    elif n_samples > 1_000_000:
        recommendation = "SGD / Mini-Batch Gradient Descent (Dataset too massive f
    else:
        recommendation = "OLS Normal Equation (Fast, exact, and zero hyperparamete

    return recommendation

# --- Function Call & Testing ---
print("Small Dataset (m=1000, d=20):   ", compare_gd_vs_ols(1000, 20))
print("High-Dimension (m=5000, d=25000):", compare_gd_vs_ols(5000, 25000))
print("Big Data (m=10M, d=50):          ", compare_gd_vs_ols(10_000_000, 50))
```

🔑 Key Takeaways

1. Scikit-Learn's `LinearRegression` uses OLS (via LAPACK SVD `scipy.linalg.lstsq`) under the hood.
2. For Big Data with millions of rows, use `SGDRegressor`.